

Multi-Source Candidate Data Transformer

1. Executive Summary

Ingesting candidate profiles from highly heterogeneous and unstructured formats (ATS JSON, resumes in PDF/DOCX, LinkedIn/GitHub links, recruiter notes) is a major hurdle in talent acquisition. The Multi-Source Candidate Data Transformer is a modular, configurable, and robust Python-based pipeline designed to normalize, reconcile, validate, and shape candidate profiles dynamically. It ensures total data isolation between executions, outputs complete provenance tracking for full auditability, and structures profiles to user-defined schemas without code modifications.

2. Core Pipeline Architecture

The system processes candidate information through five isolated, sequential lifecycle stages:

- **Ingestion & Parsing:** Extracts raw text, section markers, and field groups using specialized parsers.
- **Normalization:** Standardizes fields to global formats (lowercased emails, E.164 phone formats, ISO-3166-1 country codes, and standardized dates).
- **Merge & Conflict Resolution:** Reconciles fields. Single-value fields are resolved by source confidence; list fields are merged and deduplicated.
- **Dual Validation:** Validates canonical profile completeness and minimum overall confidence (>0.5), and checks projected outputs against schema types.
- **Custom Projection:** Maps canonical records onto target schemas using a JSON config (supporting path indexing and missing-field fallback strategies).

3. Key Engineering Innovations & Edge Case Handling

- **Tenure Overlap Protection:** Tracks experiences in a set of unique calendar months (YYYY-MM) to compute absolute tenure, preventing experience inflation from concurrent roles.
- **Agreement-Based Skill Boosting:** Consolidates duplicate skills, applying a +0.10 confidence agreement bonus for each independent source that matches the skill.
- **Granular Provenance & Safety:** Outputs audit logs (field, source, method) for complete transparency. Enforces complete execution isolation to prevent memory leaks and data contamination.

4. Validation & Verification Results

The pipeline is validated via a robust pytest suite covering edge cases (overlapping positions, malformed inputs, and custom projection schemas). The engine exports diagnostic audit trails to `output/validation_report.json`, capturing validation flags, warnings, and final pipeline status.

5. Technology Stack

- **Frontend:** Streamlit
- **Backend:** Python
- **Database:** SQLite
- **Data Processing:** JSON, CSV, PDFPlumber, Python-Docx, Pydantic
- **DevOps:** Docker, GitHub Actions